

Министерство образования и науки

Краснодарского края

ГБПОУ КК ЕПК

Отделение педагогики

Специальность 09.02.07 Информационные системы и программирование

ДОКУМЕНТИРОВАНИЕ
программного кода
составленного на языке программирования C++
в среде программирования Qt Creator
(стиль Qt)

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

г. Ейск, 2025 г.

Специальные блоки комментариев

Специальный блок комментариев - это блок комментариев в стиле C или C++ с некоторыми дополнительными пометками (тегами), которые воспринимаются Doxygen, как фрагмент структурированного текста, который должен быть включен в создаваемую документацию.

Для каждой сущности в коде существует два (или в некоторых случаях три) типа описаний, которые вместе образуют документацию для этой сущности: **краткое описание** (brief description) и **подробное описание** (detailed description). Оба не являются обязательными.

Для методов и функций существует также третий тип описания, так называемое **описание в теле** ("in body" description), которое состоит из объединения всех блоков комментариев, найденных в теле метода или функции.

Допускается наличие более одного краткого или подробного описания (но не рекомендуется, поскольку порядок, в котором описания будут отображаться, не указан).

Как следует из названия, краткое описание - это короткая однострочная информация, в то время как подробное описание содержит более длинную и подробную документацию. Есть несколько способов создания блоков подробного описания.

Оформление блока подробного описания (detailed description)

Первый вариант

Вариант оформления в стиле Qt (предпочтительный) — к комментариям в стиле C добавляется восклицательный знак. Обратите внимание, что среда программирования Qt Creator автоматизирует процесс документирования кода.

```
/*!  
 * ... текст ...  
*/
```

Промежуточные звездочки не обязательны:

```
/*!  
 ... текст ...  
*/
```

Второй вариант

Основан на использовании блока, состоящего, как минимум, из двух строк комментариев в стиле C++, где каждая строка начинается с дополнительной косой черты или восклицательного знака. Например:

```
///  
/// ... текст ...  
///
```

Или

```
///  
///  
///  
///!
```

Оформление блока краткого описания (brief description)

Для краткого описания также есть несколько возможностей. Можно использовать команду `\brief` с одним из приведенных выше блоков комментариев. Эта команда заканчивается в конце абзаца, поэтому подробное описание следует после пустой строки.

```
/*! \brief Краткое описание.  
 * Продолжение краткого описания.  
 *  
 * Здесь начинается подробное описание.  
 */
```

Возможны также несколько вариантов без использования команды `\brief`, но в сочетании блоков `brief description` и `detailed description`.

Первый вариант

```
/// Краткое описание.  
/** Подробное описание. */
```

Или

```
//! Краткое описание.  
/** Подробное описание. */
```

Второй вариант

```
//! Краткое описание.  
  
//! Подробное описание  
//! начинается здесь.
```

Обратите внимание на пустую строку в последнем примере, которая требуется для отделения краткого описания от блока, содержащего подробное описание. (Для параметра JAVADOC_AUTOBRIEF в этом случае также должно быть установлено значение NO).

Однострочные комментарии для элементов

Если нужно задокументировать элементы файла, структуры, объединения, класса или перечисления, **необходимо поместить блок документации после элемента, а не перед ним**. Для этого вам необходимо поместить дополнительный маркер < в блок комментариев. Обратите внимание, что аналогичный подход применяется и для параметров функции.

Например, в стиле Qt:

```
int var; /*!< Подробное описание после элемента */
```

Или

```
int var; /**< Подробное описание после элемента */
```

Но чаще всего требуется лишь указать краткое описание

```
int var; /*!< Краткое описание после элемента
```

Или

```
int var; ///< Краткое описание после элемента
```

Пример документирования класса

```
1.  ///  
2.  /*!  
3.  Более подробное описание класса.  
4.  */  
5.  
6.  class QTeststyle_Test  
7.  {  
8.      public:  
9.  
10. ///  
11. /*!  
12.     enum TEnum {  
13. TVal1, /*!  
14. TVal2, /*!  
15. TVal3 /*!  
16.     }  
17. ///  
18. /*!  
19. *enumPtr,  
20. ///  
21. /*!  
22. enumVar;  
23.  
24. ///  
25.     /*!  
26. Более подробное описание конструктора.  
27.     */  
28. QTeststyle_Test();  
29.  
30. ///  
31.     /*!  
32. Боле подробное описание деструктора.
```

```

33.     */
34. ~QTstyle_Test();
35.
36. ///! Обычный метод, принимающий два аргумента и возвращающий целое
значение.
37.     /*!
38.     \param a целочисленный аргумент.
39.     \param s константный указатель на символьную строку.
40.     \return Результаты тестирования
41.     \sa QTstyle_Test(), ~QTstyle_Test(), testMeToo() and publicVar()
42.     */
43.     int testMe(int a, const char *s);
44.
45.     ///! Чисто виртуальный метод.
46.     /*!
47.     \sa testMe()
48.     \param c1 первый аргумент.
49.     \param c2 второй аргумент.
50.     */
51.     virtual void testMeToo(char c1, char c2) = 0;
52.
53.     ///! Общедоступная переменная.
54.     /*!
55.     Описание.
56.     */
57.     int publicVar;
58.
59.     ///! Указатель на функцию.
60.     /*!
61.     Описание.
62.     */
63.     int (*handler)(int a, int b);
64. };

```

Так будет выглядеть в документации

Detailed Description

A test class.

A more elaborate class description.

Member Enumeration Documentation

◆ TEnum

enum `QTstyle_Test::TEnum`

An enum.

More detailed enum description.

Enumerator

TVal1	Enum value TVal1.
TVal2	Enum value TVal2.
TVal3	Enum value TVal3.

Detailed Description

A test class.

A more elaborate class description.

Member Enumeration Documentation

◆ TEnum

enum `QTstyle_Test::TEnum`

An enum.

More detailed enum description.

Enumerator

TVal1	Enum value TVal1.
TVal2	Enum value TVal2.
TVal3	Enum value TVal3.

Constructor & Destructor Documentation

◆ `QTstyle_Test()`

`QTstyle_Test::QTstyle_Test ( )`

A constructor.

A more elaborate description of the constructor.

Детальное описание метода класса

Для подробного описания методов могут применяться следующие специальные команды:

- `\param` - Параметры передаваемые функции. Пример:

```
\param name Имя пользователя
```

Может применяться с опциональным атрибутом «направление». Этот атрибут может иметь три значения: `[in]`, `[out]`, `[in, out]`. Например:

```
/*!  
Копирует содержимое из исходной области памяти в целевую область памяти  
\param[out] dest Целевая область памяти  
\param[in] src Исходная область памяти  
\param[in] n Количество байтов, которые необходимо скопировать  
*/  
void memcpy(void *dest, const void *src, size_t n);
```

- `\return` - Возвращаемое функцией значение. Пример:

```
\return Информация о пользователе
```

- `\throw` - Исключения выбрасываемые функцией. Пример:

```
\throw DatabaseError Если произошла ошибка при подключении к базе данных
```

- `\sa` - Начинается абзац, в котором может быть указана одна или несколько перекрестных ссылок на классы, функции, методы, переменные, файлы или URL.

Оформление шапки кода

Главный файл программы обычно содержит «шапку» с подробным описанием проекта (не обязательно, если эта информация документируется в отдельном файле).

```

1.  /*!
2.   * \brief Pretty nice class.
3.   * \details This class is used to demonstrate a number of section
      commands.
4.   * \author John Doe
5.   * \author Jan Doe
6.   * \version 4.1a
7.   * \date 1990-2011
8.   * \pre First initialize the system.
9.   * \bug Not all memory is freed when deleting an object of this
      class.
10.  * \warning Improper use can crash your application
11.  * \copyright GNU Public License.
12.  */
13.  class SomeNiceClass {};

```

Так будет выглядеть в документации

Pretty nice class.

This class is used to demonstrate a number of section commands.

Author

John Doe

Jan Doe

Version

4.1a

Date

1990-2011

Precondition

First initialize the system.

Bug

Not all memory is freed when deleting an object of this class.

Warning

Improper use can crash your application

Copyright

GNU Public License.

Структурные команды

Иногда удобнее размещать документацию в другом месте. Doxygen позволяет размещать блоки документации практически где угодно (за исключением тела функции или обычного блока комментариев в стиле C).

1. Блок документации, который не размещается непосредственно перед (или после) элемента, содержит структурные команды внутри блока документации. **Это может привести к дублированию информации.** Поэтому на практике следует избегать использования структурных команд, если только этого не требуют исключительные обстоятельства.
2. Структурные команды начинаются с обратной косой черты (\). Далее следует имя команды и один или несколько параметров.

К структурным командам относятся:

- \struct для документирования C-структуры.
- \union для документирования объединения.
- \enum для документирования типа перечисления.
- \fn для документирования функции.
- \var для документирования переменной, typedef или значения перечисления.
- \def для документирования #define.
- \typedef для документирования определения типа.
- \file для документирования файла.
- \namespace для документирования пространства имён.
- \package для документирования пакета Java.
- \interface для документирования интерфейса IDL.

Применять структурные команды — не рекомендуется!

Подробные сведения и примеры оформления структурных команд см. в документации: <https://www.doxygen.nl/manual/docblocks.html>

Страницы документации

- Документирование кода: <https://www.doxygen.nl/manual/docblocks.html>
- Настройка выходных данных:
<https://www.doxygen.nl/manual/customize.html>
- Дополнительная документация:
<https://www.doxygen.nl/manual/additional.html>
- Выходные форматы: <https://www.doxygen.nl/manual/output.html>

- Автоматическая генерация ссылок:
<https://www.doxygen.nl/manual/autolink.html>
- Поддержка Markdown: <https://www.doxygen.nl/manual/markdown.html>
- HTML — команды: <https://www.doxygen.nl/manual/htmlcmds.html>

Документирование классов QML

Файлы QML можно документировать в системе Doxygen с помощью фильтра Doxyqml. Он интегрируется как фильтр ввода Doxygen, превращая файлы .qml в псевдо-C++, который Doxygen может использовать для генерации документации.

Установка Doxyqml

```
sudo dnf install pip3
pip3 install doxyqml
```

Настройка Doxygen

Чтобы настроить Doxygen на использование Doxyqml, необходимо внести несколько изменений в файл конфигурации Doxygen.

Добавьте расширение .qml к ключу FILTER_PATTERNS:

```
FILTER_PATTERNS = *.qml=doxyqml
```

Примечание: в Windows Doxyqml устанавливается в папку Scripts вашей установки Python. Если эта папка не находится в PATH, добавьте её или используйте полный путь к Doxyqml (но это менее удобно для разных компьютеров)

Добавьте расширение .qml к FILE_PATTERNS:

```
FILE_PATTERNS = *.qml
```

Начиная с Doxygen 1.8.8, вы также должны добавить расширение .qml в EXTENSION_MAPPING:

```
EXTENSION_MAPPING = qml=C++
```

Документирование типов. Функции

QML частично типизирован: функции нетипизированы, а свойства и сигналы типизированы. Doxqml позволяет определять типы, если они отсутствуют или недостаточно точны.

Типы в документации можно определить следующим образом:

```
1.  /*!
2.    * Create a user
3.    * \param type:string firstname User firstname
4.    * \param type:string lastname User lastname
5.    * \param type:int User age
6.    * \return type:User The User object
7.    */
8.  function createUser(firstname, lastname, age);
```

Свойства

Свойства QML типизированы, поэтому Doxqml использует их по умолчанию. Тем не менее вы можете переопределить тип, используя тот же синтаксис `type:<name>`. Это полезно для документирования псевдонимов свойств:

```
1.  /*! type:string The user lastname */
2.  property alias lastname: someObject.text
```

Сигналы

Сигналы QML типизированы, поэтому нет необходимости использовать синтаксис `type:<name>` для документирования их параметров. Использование синтаксиса `type:<name>` в документации сигналов не работает: Doxqml не удалит его, а Doxugen перепутает его с именем параметра.

```
1.  /**
2.    * User just logged in
3.    * @param user The user which logged in
4.    */
```

```
5. signal loggedIn(User user)
```

Извлечение внутренних элементов

Элементы QML с идентификатором экспортируются как приватные переменные-члены. Если вы установите для ключей EXTRACT_ALL и EXTRACT_PRIVATE Doxygen значение YES, то эти элементы будут отображаться в сгенерированной документации.